NYCU 國立陽明交通大學
自主及適應性系統實驗室
Autonomous and Adaptive Systems Laboratory

Homework Assignment 3: Players for Prisoners Dilemma

CSIC30122: Games Theory and Its Applications

Professor: Li-Hsing Yen
TA: YAN-WEI CHEN, CHUN-YU LIN

DeadLine: 2026-0512-2359

Institute of Computer Science and Engineering
National Yang Ming Chiao Tung University

AASLAB

Outline

- Overview - Repeated Prisoners Dilemma Game
- Opponent Strategy
- Baseline: Random Protagonist Agent
- Goal: Designing Your Protagonist Agent
- Requirement
- Scoring Policy
  - Submission
- References

Overview

Implementing the Learning Automata (LA) and Reinforcement Learning (RL)

Protagonist Agents for a Finitely Repeated Game

- Getting familiar with Learning Automata
  - Based on reward history
- Integrating Reinforcement Learning into matrix game
  - Based on state and reward history
- Observing difference between LA and RL protagonist agents

Write a report to briefly describe your code and summarize your analysis, as well as answer the free response questions

- Summarizing into one report

Repeated Prisoners Dilemma in This Assignment

- Prisoners Dilemma with a Limited Number of Stages, e.g., **100 stages**
  - Stage game: static game (identical game matrix)
  - Incomplete information: players are unaware of the game matrix in advance
  - Perfect game: players observe the result of previous stages
  - Definition: a_p^i and a_o^i are **i**-th stage **actions** w.r.t **p**rotagonist and **o**pponent

Opponent

Protagonist

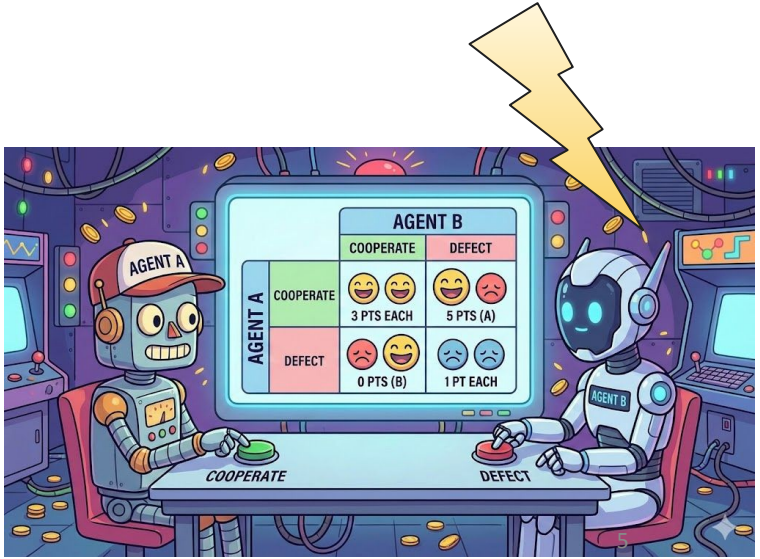
	Silent (S)	Betrayal (B)
Silent (S)	(10, 10)	(4, 12)
Betrayal (B)	(12, 4)	(6, 6)

Goal: Maxmizing total payoff of 100-stage Prisoners Dilemma

Opponent Strategy

- There is an opponent agent with the following four fixed-strategy:
  a. Random
  b. Always Betrayal ($a_o^i = B, \forall i$)
  c. Always Silent ($a_o^i = S, \forall i$)
  d. Tit for tat ($a_o^i = a_p^{i-1}, \forall i > 1$)

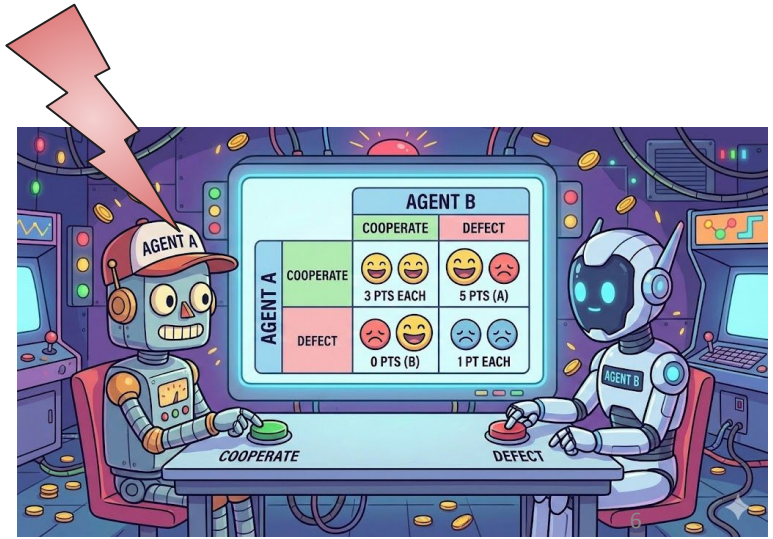
In each play of a repeated game, opponent only chooses one strategy and repeatedly play with this strategy until the end



Baseline: Random Protagonist Agent

- Randomly choosing action to play against the opponent (fixed strategy) in repeated Prisoners Dilemma game

This will be baseline reference results

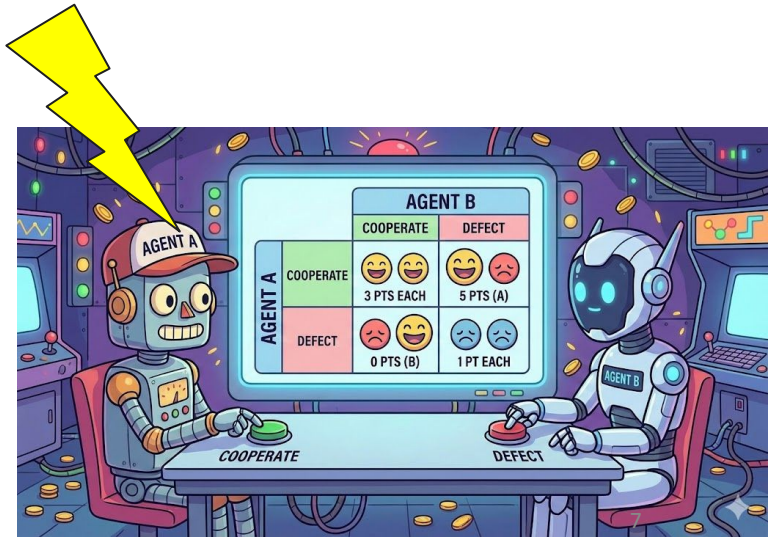


Designing Your Protagonist Agents

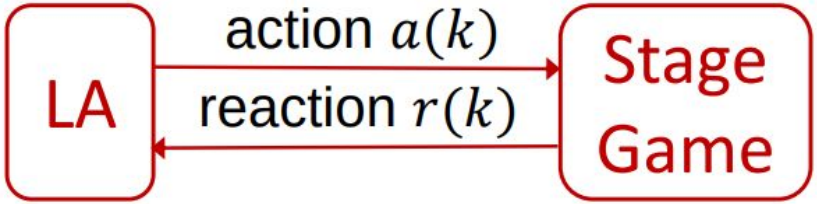
Protagonist agent via:

- Learning Automata (LA)
- Reinforcement Learning (RL)

Your protagonist agents should act on 4 fixed strategies of opponent! (namely, Random, Always selecting Betrayal, Always selecting Silent, Tit for tat)



Learning Automata (LA)



Lec. repeated_games p. 43 ~ p. 62:

- Penalty consideration: L_{R-I} vs. L_{R-P}
- Generalization: Pure-Binary vs. Continuous

We can summarize the concept into following equations:

Prob. of selected action i

$$p_i(k+1) \leftarrow p_i(k) + \alpha\beta(k)(1 - p_i(k)) - \rho(1 - \beta(k))p_i(k)$$

Prob. of other actions j

$$p_j(k+1) \leftarrow (1 - \alpha \cdot \beta(k))p_j(k) - \rho(1 - \beta(k))p_j(k) + \frac{\rho(1 - \beta(k))}{N_a - 1}$$

, for all $j \neq i$

Learning Automata (cont.)

Lec. repeated_games p. 43 ~ p. 62:

- Penalty consideration: $LR\text{-}I$ vs. $LR\text{-}P$
- Generalization: Pure-Binary vs. Continuous

Summarizing the concept into following equations:

Pure-Binary:

$$\beta(k) = \begin{cases} 1, & \text{if } r(k) > \bar{R}(k) \\ 0, & \text{if } r(k) \leq \bar{R}(k) \end{cases}$$

success

fail

Continuous:

$$\beta(k) = \frac{r(k) - r_{min}}{r_{max} - r_{min}}$$

Prob. of selected action i

$$p_i(k+1) \leftarrow p_i(k) + \alpha\beta(k)(1 - p_i(k)) - \rho(1 - \boxed{\beta(k)})p_i(k)$$

Prob. of other actions j

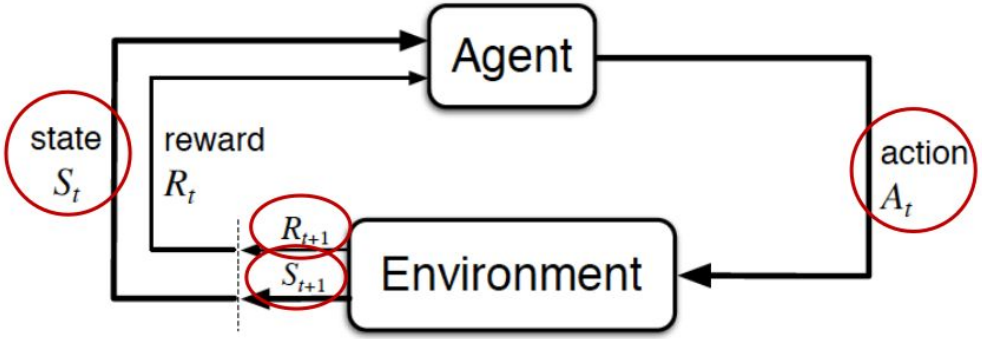
$$p_j(k+1) \leftarrow (1 - \alpha \cdot \beta(k))p_j(k) - \boxed{\rho}(1 - \beta(k))p_j(k) + \frac{\rho(1 - \beta(k))}{N_a - 1}$$

, for all $j \neq i$

"0" implying $LR\text{-}I$, otherwise $LR\text{-}P$

Reinforcement Learning (RL)

- Model the problem as a Markov Decision Process (MDP)



- Use reinforcement learning (RL) to maximize accumulated reward (payoff)

Game Theory	Reinforcement Learning
game	environment
player	agent
pure strategy	action
mixed strategy	policy
strategy profile	joint action (action profile)
utility; payoff	reward

Lec. repeated_games p. 63

Repeated Prisoners Dilemma Environment

Observation State: [Protagonist's Last Action, Opponent's Last Action]

Action: {0 (silent), 1 (betrayal)}

Rewards: following game matrix

	Silent (S)	Betrayal (B)
Silent (S)	**(10, 10)**	**(4, 12)**
Betrayal (B)	**(12, 4)**	**(6, 6)**

Initial State: [-1, -1]

Episode end: 100 time steps (repeated-game stages)

- Hence, maximum total score of one episode is 1200

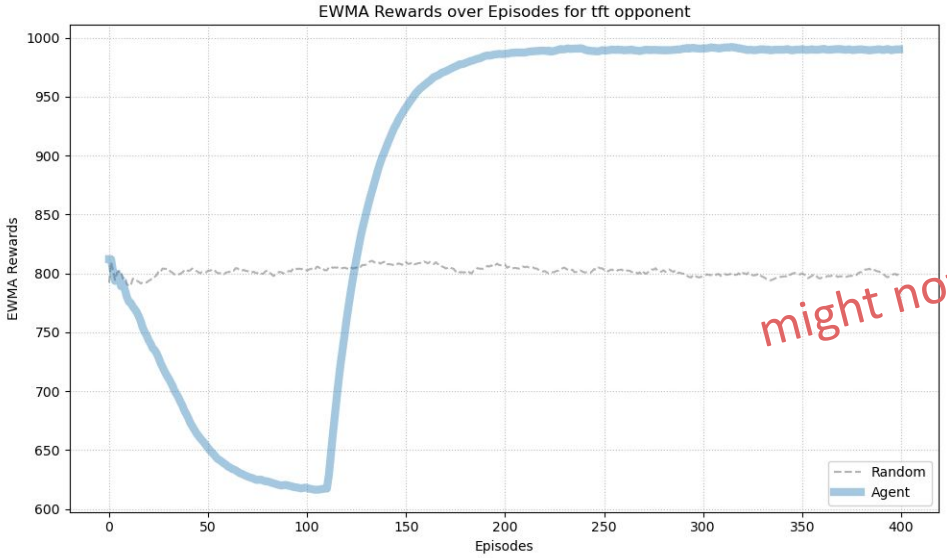
What to Do?

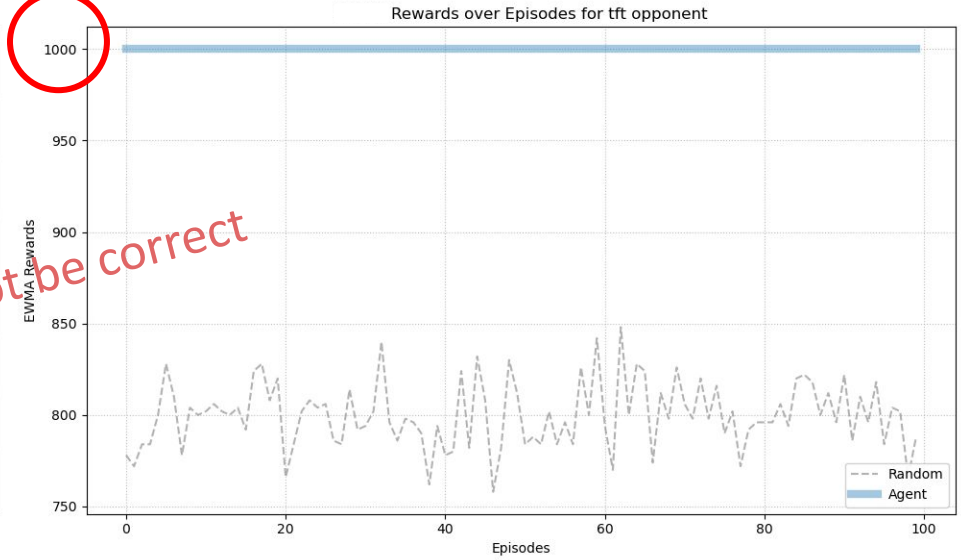
Implementing **both** Learning Automata **and** Reinforcement Learning

- Learning Automata:
  - $LR{-}I$ or $LR{-}P$ (arbitrarily select)
  - Pure-Binary or Continuous (arbitrarily select)
- RL can use Stable Baselines3 (SB3) or others, but should generate identical output format
  - e.g. might need monitor (stable_baselines3.common.monitor) and certain callback (stable_baselines3.common.callbacks) to record rewards during training for SB3
- **Both <u>protagonist agents</u> need to act on <u>4 fixed opponent strategies</u>**
  - **In each trial, opponent sticks to one of 4 strategies until the end**

TFT Opponent

sum of 100-stage reward





might not be correct

This is an example reward curve of TFT-strategy opponent with Random protagonist and stable_baselines3 DQN protagonist for training (left) and inference (right)

Attachment

Attached files:

- PrisonersDilemmaEnv class **(should not modify)**
  - gymnasium:1.0.0
- BaseAgent and Baseline RandomAgent **(should not modify)**
- Plotting facility **(should not modify)**
- LAAgent (LA protagonist need to be completed)
- Main program for training and evaluation (might minorly modify)
  - Invoke your protagonist agents, store the output results of each agent, plot the results
  - Can print any messages, but cannot modify the main logic

Try to run main.py first!

Check what the program does.

Requirement (1/2)

- Completing LA logic
- Developing your RL agent (implemented in another new .py file)
- TAs will just execute the main program
  - 2 Developed-protagonist (LA and RL) should be invoked
    - Outputing the raw reward (checking what RA do in main program)
  - Plotting the results

Requirement (2/2)

- Answering 4 Free response questions
  - Do you observe any **interesting phenomena** when your agents (LA or RL agent) play this game?
  - Do you adopt any **tricks** in the agent, or just the vanilla algorithm?
  - Do you consider states are **necessary** for an agent to make decisions? (LA w/o state, but RL w/ state) Or is it necessary in **what conditions**?
  - In MDP of **standard** Reinforcement Learning, "State" and "Action" are distinct concepts. However, in our Repeated Prisoner's Dilemma, the observation state is explicitly defined as the previous actions [Protagonist's Last Action, Opponent's Last Action]. <u>Why is it necessary to formulate previous actions as the current state?</u>

Scoring Policy

A. Comparing developed protagonist with Random protagonist (**plotting**, 64 points)
   - Four types of opponent:
     - Random (10 points for LA, 6 points for RL)
     - Always-Betrayal (10 points for LA, 6 for RL)
     - Always-Silent (10 points for LA, 6 for RL)
     - Tit for tat (10 points for LA, 6 for RL)
   - <u>Not highly focusing on performance</u>
     - Just trying to be better than Random protagonist
     - (No worries if you can't make it; Not a strict grading criterion)
   - <u>Observing which protagonist has poor performance against which opponent</u>

B. Free response questions (36 points, 9 points each)

Scoring Policy

Writing the report in either English or Mandarin

- Source code explanation and **AI prompt (if you use)** description
- Results analysis and Free Response Questions
  - **Attaching plotting in report for both training and inference**
  - **0 points for no plotting in report**

NO POINT: Plagiarism in source code or report

Strict 30% off deduction for **Late Turning In**

~ No matter when after the deadline

Submission

Submitting a zip file to E3 platform with following structure:

- ✓ <your student ID>_HW3.zip (e.g., 412551007_HW3.zip)
  - ➢ <your student ID>_HW3_report.pdf
  - ➢ src/
    - main.py
    - ……(other necessary files, including provided ones)

Strict 10% off deduction for **Wrong Format**

Appendix